

9 КЛАС · РОЗДІЛ 3 · УРОКИ 19–20

Сортування і пошук у списках

§ 3.3, стор. 117–125

Сортують не для краси — сортують, щоб потім знайти швидко

Одна різниця — в одному знаку

- за зростанням: кожне БІЛЬШЕ за попереднє $a[i+1] > a[i]$
- за спаданням: кожне МЕНШЕ $a[i+1] < a[i]$
- за неспаданням: більше АБО ДОРІВНЮЄ $a[i+1] \geq a[i]$
- за незростанням: менше АБО ДОРІВНЮЄ $a[i+1] \leq a[i]$
- Список $[3, 3, 5]$ — за неспаданням. Бо $3 > 3$ неправда

sort псує оригінал, sorted — ні

- `b = sorted(a)` — робить НОВИЙ список, а цілий
- `a.sort()` — сортує САМ список, оригіналу більше немає
- За спаданням: `reverse = True` в обох
- `sorted` — коли початковий порядок ще потрібен
- `sort` — коли він більше не потрібен

TOP

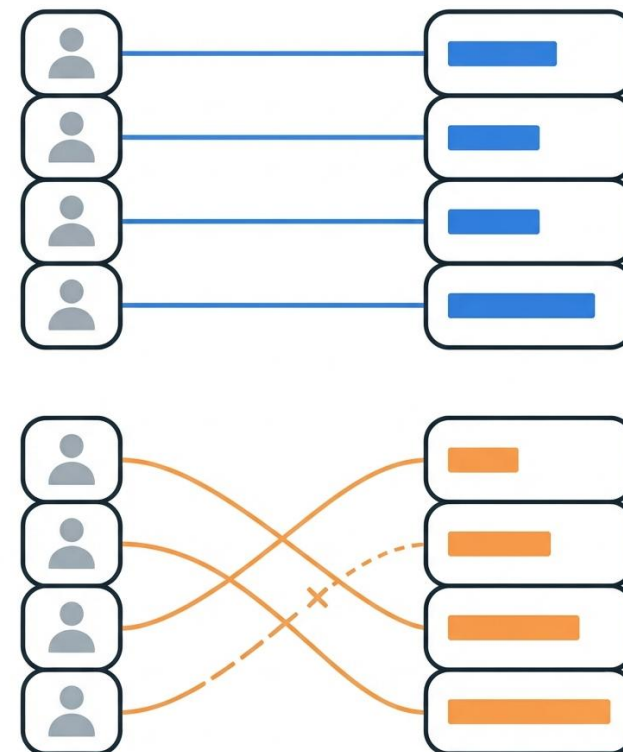


sort нічого не повертає

- `e = a.sort()` → `e` стає `None`
- Тому `a = a.sort()` знищує список
- І помилка буде `HE` там, де причина — `a` рядком нижче
- Правило: впало на списку → дивіться вище, чи не було `= ...sort()`

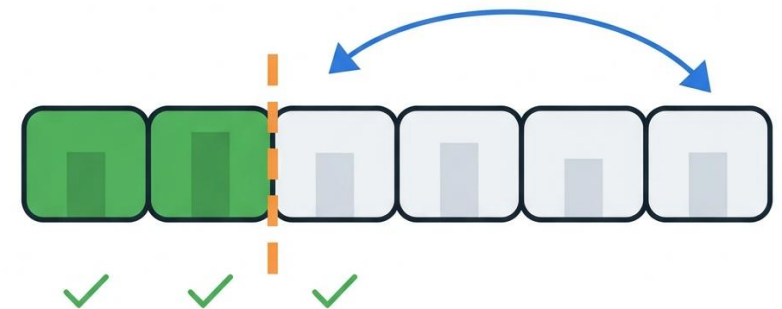
Коли sort дає неправду

- Два зв'язані списки: прізвища й час бігу
- `chas.sort()` відсортує час...
- ...а прізвища залишить на місці
- І тепер Коваль «пробіг» за 12,4 — неправда, зроблена правильним кодом
- У своєму алгоритмі обмін пишемо МИ — і міняємо обидва списки



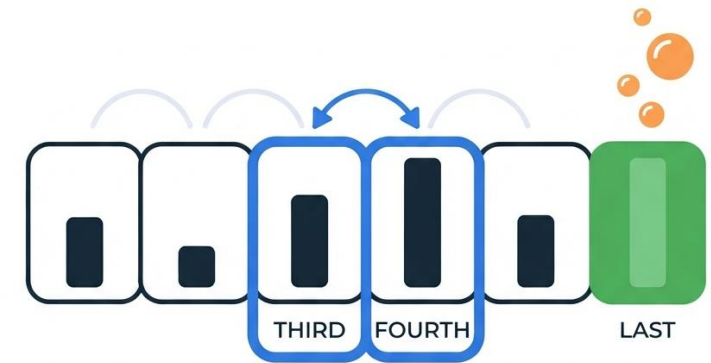
Знайти найменший — обміняти — повторити

- Найменший у ще НЕвідсортованій частині
- Обміняти його з ПЕРШИМ елементом цієї частини
- Лівий край відсортованої частини зсувається на один
- Для 6 чисел — 5 кроків, не 6
- На останньому кроці два елементи стають на місця одночасно



Порівнюємо тільки сусідів

- Беремо двох сусідів. Лівий більший — міняємо
- Потім наступну пару. І так до кінця
- Після 1-го проходу найбільший уже в кінці — і не рухається
- Тому кожен наступний прохід КОРОТШИЙ: $\text{range}(n - 1 - i)$
- Це не оптимізація для краси. Це частина алгоритму

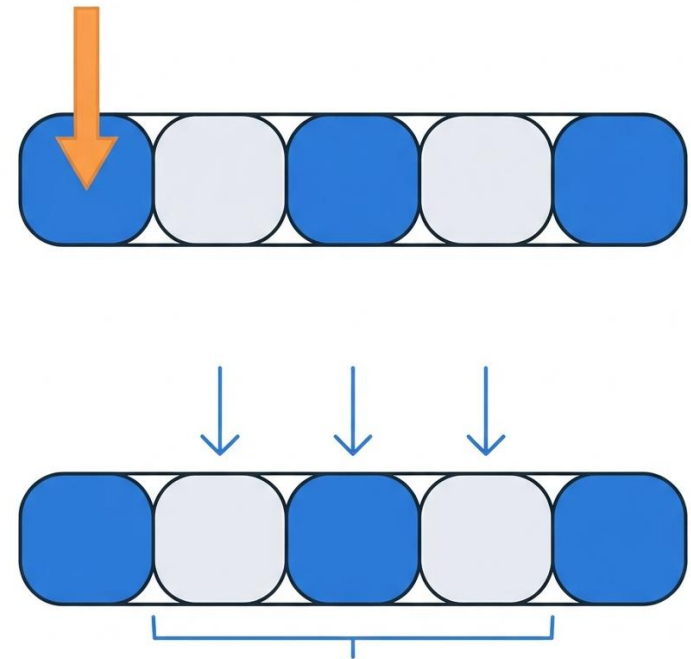


Як НЕ обміняти два значення

- $x = 5, y = 9$. Робимо $x = y$, потім $y = x$
- Стало $x = 9, y = 9$. П'ятірка ЗНИКЛА
- Бо перша команда затерла старе значення x
- Правильно: $t = x; x = y; y = t$
- Або по-пайтонівськи: $x, y = y, x$

Три питання — три інструменти

- Чи $\in \rightarrow x \text{ in } a$
- Скільком дорівнює $\rightarrow a.\text{count}(x)$
- Де саме, ВСІ місця $\rightarrow$ цикл
- `index` дасть лише ПЕРШЕ місце
- Перевірка: `count` і кількість індексів мусять збігтися



Другий варіант має ДВА дефекти

- `m = mini(a[first:])` — функції `mini` немає, вбудована зветься `min`
- Це видно НАВІТЬ на папері → `NameError`
- `ind = a.index(m)` — шукає по ВСЬОМУ списку, а не з місця `first`
- Цього не видно ніяк

Правильне рішення, застосоване не думаючи

- У першому варіанті `mini` — це ЗМІННА з мінімумом
- Названа `mini`, а не `min` — щоб не затінити вбудовану. Правильно!
- Це рівно те, про що ми говорили на уроці 17
- А потім це ім'я потрапило на місце ВИКЛИКУ вбудованої
- Навіть правильне рішення ламає код, якщо застосувати його підряд

Помилки немає, список не відсортований

- [23, 7, 4, 16, -2, 10] → правильно
- [1, 3, 1] → [3, 1, 1]
- [7, 3, 3, 9, 3] → [7, 9, 3, 3, 3]
- min шукає в частині, а index — у всьому списку
- Виправлення — одне число: `a.index(m, first)`

МЕТОДИ ВИБОРУ Й ОБМІНУ (СОРТУВАННЯ)

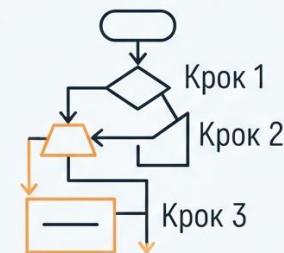
Можна не малювати картинки

Оскільки є відео AlgoRythmics

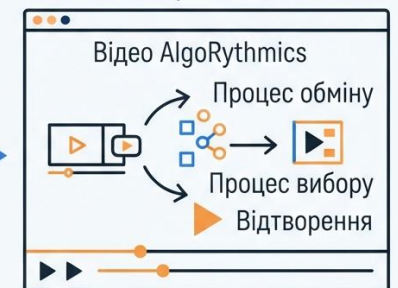
Посилання — у звірці й заготовці

Це краще за схему

Статична Схема



Алгоритміка
Відеовізуалізація
Перегляд



Краще розуміння
Динамічне спостереження

«Чи правильно, якщо є рівні числа?»

- Перевірено на шести списках з однаковими числами
- варіант 1 (цикл по j) — 0 збоїв із 6
- варіант 2 (з `index`) — 4 збої із 6
- метод обміну — 0 збоїв
- Книжка поставила гарне питання. І сама на нього попалася

Прискорення залежить від ДАНИХ

- Додали перевірку: не було обмінів — вихід
- випадковий список: 5 проходів $\rightarrow$ 4
- уже відсортований: 5 $\rightarrow$ ОДИН
- перевернутий: 5 $\rightarrow$ 5, не зекономили нічого
- Той самий код на різних даних працює по-різному

Що взяти з цих двох уроків

- `sorted` робить копію, `sort` псує оригінал
- `sort` повертає `None` — не присвоюй його списку
- Зв'язані списки обмінюй ЗАВЖДИ разом
- `index` дає перше входження — не завжди те, що потрібне
- Сортують, щоб потім знайти швидко